COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Sep 17, 2026	**DATE SUBMITTED:**	Oct 6, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 9 **Tree ADT**

1. PROCEDURE OUTPUT

Code

```
template <typename T>
class GeneralSearchTree{
    private:
        T key;
        GeneralSearchTree<T>* firstChild;
        GeneralSearchTree<T>* nextSibling;

    public:
        //constructor:
        GeneralSearchTree(T value);

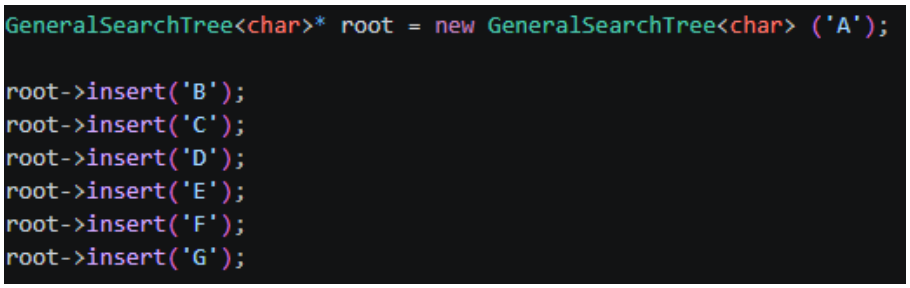
        //insertion
        void insert(T value);

        //traversal:
        void preOrder();
        void inOrder();
        void postOrder();



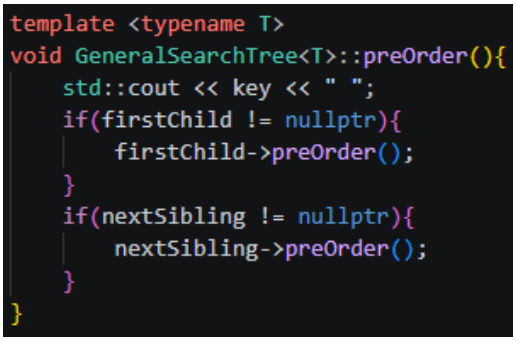
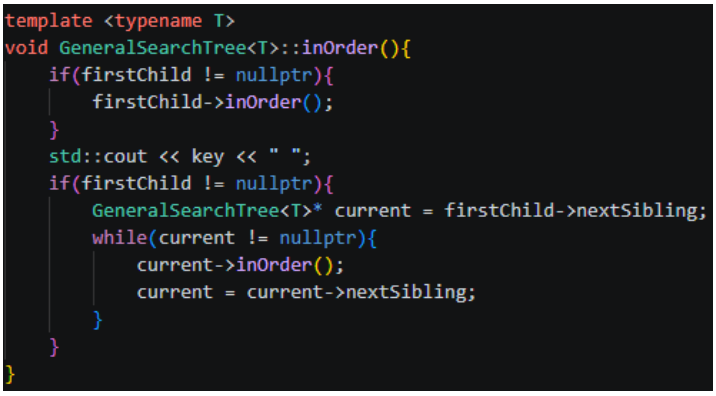
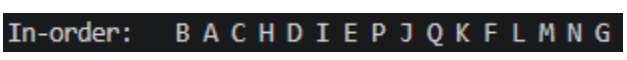
};

template <typename T>
GeneralSearchTree<T>::GeneralSearchTree(T value){
    key = value;
    firstChild = nullptr;
    nextSibling = nullptr;
}
```

	
Comments	Since this tree does not have a maximum of 2 children per node, it does not follow the comparison of values to determine its position in a tree data structure. In my understanding, you locate where you want to place the data, and that's where you will insert it.

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	2
P	0	3
Q	0	3

HANDWRITTEN:	
Pre-Order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
In-Order	B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G
Post-Order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A

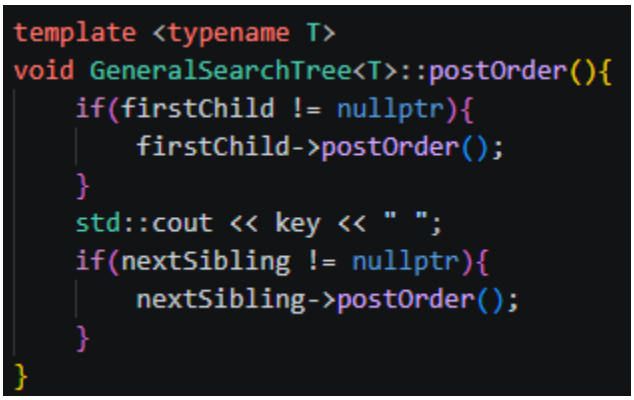
Pre-Order	
Code	
Output	
Observations	Using the binary search traversal, I made it so that it prints the root, then the first child, then lastly the remaining children. It follows the same root->left->right. Since this is a general tree, you cannot compare values as you would in a binary tree.
In-Order	
Code	
Output	
Observations	Just like the binary in-order traversal, this method sticks to the left side until all left or first child

```
template <typename T>
void GeneralSearchTree<T>::preOrder(){
    std::cout << key << " ";
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    if(nextSibling != nullptr){
        nextSibling->preOrder();
    }
}
```

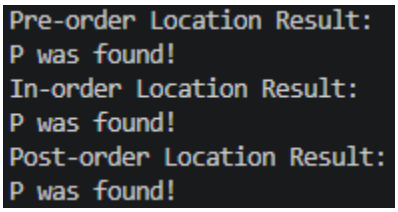
```
Pre-order:  A B C D H E I J P Q F K L M G N
```

```
template <typename T>
void GeneralSearchTree<T>::inOrder(){
    if(firstChild != nullptr){
        firstChild->inOrder();
    }
    std::cout << key << " ";
    if(firstChild != nullptr){
        GeneralSearchTree<T>* current = firstChild->nextSibling;
        while(current != nullptr){
            current->inOrder();
            current = current->nextSibling;
        }
    }
}
```

```
In-order:  B A C H D I E P J Q K F L M N G
```

	values are printed. Then it prints the root as it would in a binary tree. Although the binary tree only has left and right children, general trees have multiple. In this general tree, after printing the root, the rest of the children are then printed.
Post-Order	
Code	
Output	
Observations	When it comes to post-order traversal in a general tree, like in a binary tree, the bottom nodes are printed first before the root or parent node ever gets printed. It starts from bottom to top. That's why it prints out the children first before the parent or root node.

Create a function called findData	
Code	

Output	
Observations	For this function, I created separate functions when it came to traversal. If the value came up empty, then no output would be printed. Though, if a node containing the key needed is assigned to the pointer and the pointer is evaluated, then it would be a success.

Insertion of O in Node G	
Code	

```
template <typename T>
GeneralSearchTree<T>* GeneralSearchTree<T>::findNode(int choice, T key){
    GeneralSearchTree<T>* found = nullptr;
    switch (choice){
        case 1:
            found = preFind(key);
            break;
        case 2:
            found = inFind(key);
            break;
        case 3:
            found = postFind(key);
            break;
        default:
            break;
    }
    return found;
}
```

```
//Inserting O under node G
std::cout<<"Pre-order Insertion Result: "<<std::endl;
root->findNode(1, 'G')->insert('O');
root->preOrder();
std::cout << "\n";

// std::cout<<"In-order Insertion Result: "<<std::endl;
// root->findNode(2, 'G')->insert('O');
// root->inOrder();
// std::cout << "\n";

// std::cout<<"Post-order Insertion Result: "<<std::endl;
// root->findNode(3, 'G')->insert('O');
// root->postOrder();
// std::cout << "\n";
```

Output

```
Pre-order Insertion Result:
A B C D H E I J P Q F K L M G N O
```

```
Pre-order Insertion Result:
A B C D H E I J P Q F K L M G N O
In-order Insertion Result:
B A C H D I E P J Q K F L M N G O O
Post-order Insertion Result:
B C H D I P Q J E K L M F N O O O G A
```

Observations	I created another function that would return a node so that I could insert a value as its child node. Here I used a pre-order traversal. If I were to use all 3, it would still work.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1 : Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

Code

```
#include <iostream>
#include "BST.h"

int main(){
    BinarySearchTree<int>* tree = nullptr;
    char choice;
    int value;

    while(true){
        std::cout << "Binary Tree Menu: \n\n"
                  << "A) Insert Value\n"
                  << "B) Pre-Order Traversal\n"
                  << "C) In-Order Traversal\n"
                  << "D) Post-Order Traversal\n"
                  << "E) Exit\n"
                  << "Choice: ";
        std::cin >> choice;
        switch(choice){
            case 'A':
                char userChoice;
                while(true){
                    std::cout << "Creating.. (Enter -1 to stop)" << std::endl;
                    if(tree == nullptr){
                        std::cout << "No root yet\nEnter value: ";
                        std::cin >> value;
                        tree = new BinarySearchTree(value);
                    }else{
                        std::cout << "Enter value: ";
                        std::cin >> value;
                        if(value == -1)break;
                        tree->insert(value);
                    }
                }
                break;
```

```
            case 'B':
                std::cout << "Pre-Order Traversal: \n";
                tree->preOrder();
                std::cout << std::endl;
                break;
            case 'C':
                std::cout << "In-Order Traversal: \n";
                tree->inOrder();
                std::cout << std::endl;
                break;
            case 'D':
                std::cout << "Post-Order Traversal: \n";
                tree->postOrder();
                std::cout << std::endl;
                break;
            case 'E':
                return 0;
            default:
                std::cout << "Invalid, Try Again\n";
                break;

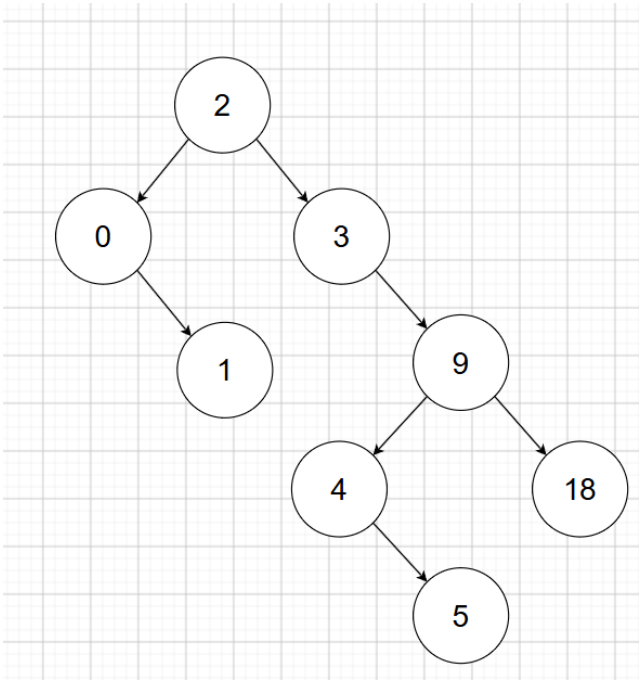
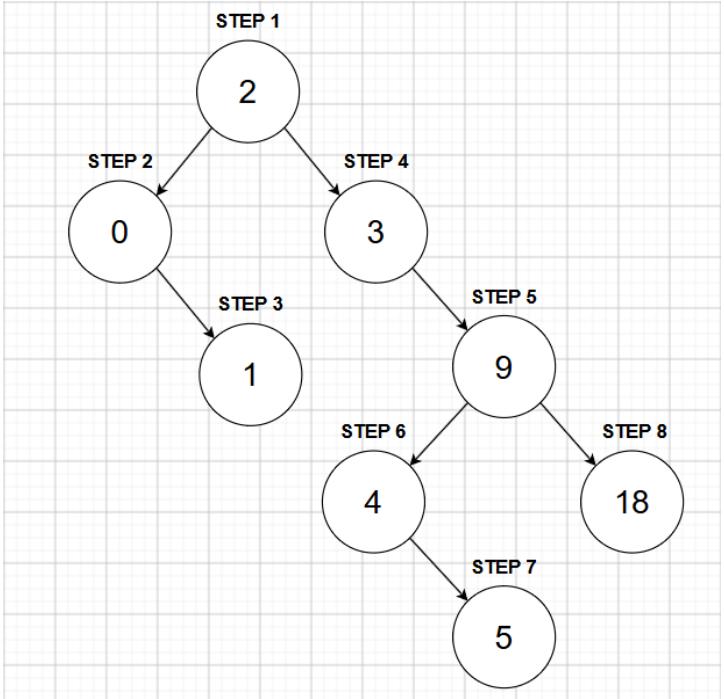
        }
    }
    return 0;
}
```

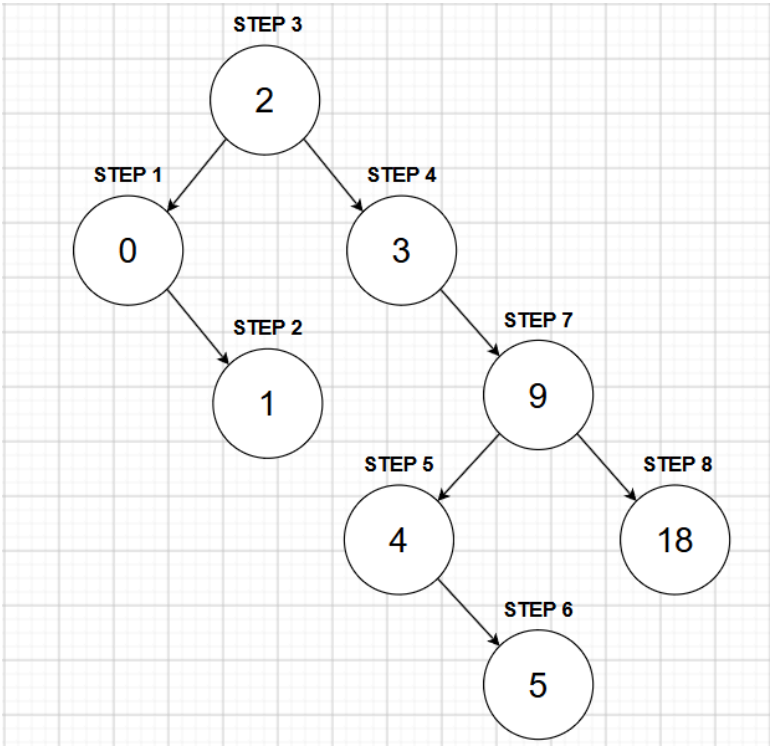
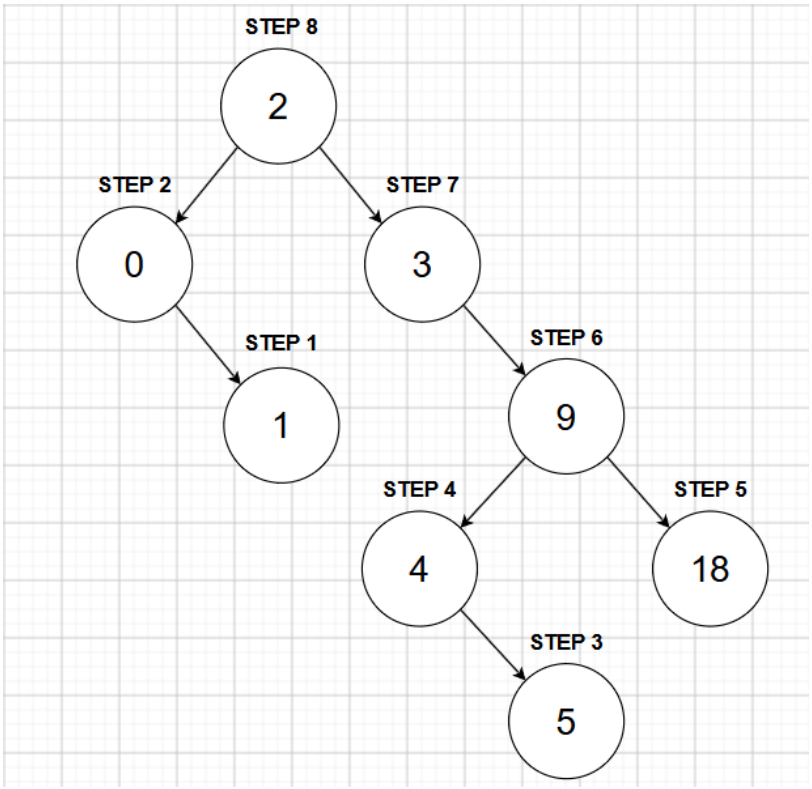
Output

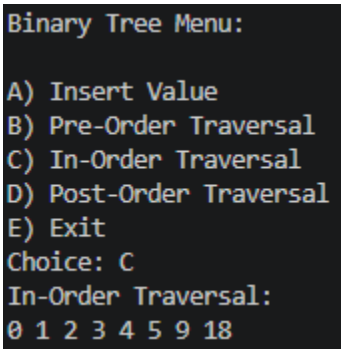
```
Binary Tree Menu:

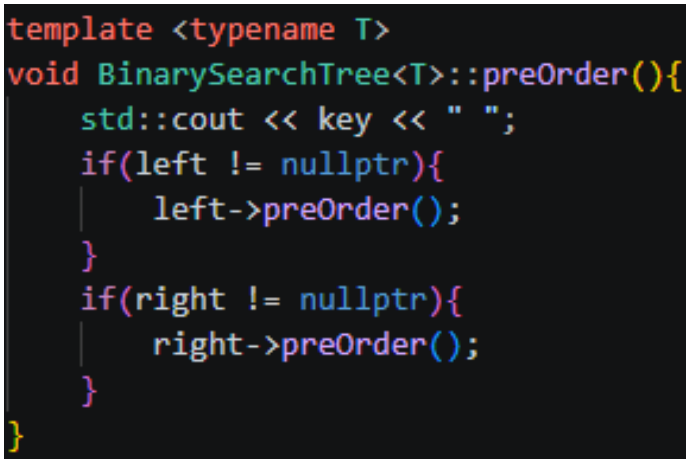
A) Insert Value
B) Pre-Order Traversal
C) In-Order Traversal
D) Post-Order Traversal
E) Exit
Choice: A
```

```
Creating.. (Enter -1 to stop)
No root yet
Enter value: 2
Creating.. (Enter -1 to stop)
Enter value: 3
Creating.. (Enter -1 to stop)
Enter value: 9
Creating.. (Enter -1 to stop)
Enter value: 18
Creating.. (Enter -1 to stop)
Enter value: 0
Creating.. (Enter -1 to stop)
Enter value: 1
Creating.. (Enter -1 to stop)
Enter value: 4
Creating.. (Enter -1 to stop)
Enter value: 5
Creating.. (Enter -1 to stop)
Enter value: -1
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.	
Tree Diagram	
Pre-Order	

In-Order	
Post-Order	

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?	
Code	
Output	

Pre-Order Traversal	
Code	

Output

```
Binary Tree Menu:

A) Insert Value
B) Pre-Order Traversal
C) In-Order Traversal
D) Post-Order Traversal
E) Exit
Choice: B
Pre-Order Traversal:
2 0 1 3 9 4 5 18
```

Post-Order Traversal

Code

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }
    if(right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

Output

```
Binary Tree Menu:

A) Insert Value
B) Pre-Order Traversal
C) In-Order Traversal
D) Post-Order Traversal
E) Exit
Choice: D
Post-Order Traversal:
1 0 5 4 18 9 3 2
```

Conclusion

In this activity, I learned that a tree stores data in levels instead of in a straight line like a list or a stack. I also learned that a general tree is different from a binary tree because a node can have any number of children, so left and right pointers are not enough. In my code, I solved this by giving each node a firstChild and a nextSibling pointer, so the children of a node form a linked list. This means insert has to start at firstChild and keep moving through nextSibling until it reaches the last child. I also learned the three traversals: pre-order visits the node first, post-order visits the children first, and in-order sits in between. Since in-order has no standard definition for a general tree, I used the rule of visiting the first child, then the node, then the remaining children.

For the procedure, I built the tree in the figure by inserting B to G under A, then using findNode to get D, E, F, G, and J so I could insert their children. My handwritten results in Table 9-3 matched the console output for all three traversals. For example, pre-order printed A B C D H E I J P Q F K L M G N. This shows that my functions follow the same steps I used by hand. The post-order function works by printing the key between the firstChild call and the nextSibling call. In-order needed a loop because calling inOrder on the first child would also run through all of its siblings. For findData, the switch uses CHOICE to pick preFind, inFind, or postFind, and it only prints "{KEY} was found!" when the returned pointer is not nullptr. Searching for P printed "was found!" for all three methods, so each traversal reaches every node and only the order of checking is different.

In the supplementary activity, I built a binary search tree using the values 2, 3, 9, 18, 0, 1, 4, and 5 through a menu-driven program. Because it is a binary search tree, each value goes left if it is smaller and right if it is larger, so 2 became the root, with 0 and 1 on the left and 3, 9, 18, 4, and 5 on the right. The in-order output was 0 1 2 3 4 5 9 18, which is sorted because the left side is always visited before the node. The pre-order output was 2 0 1 3 9 4 5 18 and the post-order output was 1 0 5 4 18 9 3 2, and both match the step numbers I placed on my diagrams. This means my diagrams and my code agree with each other. It also shows the main difference from the general tree, since the binary search tree decides the position by comparing values, while the general tree inserts a node under the parent I choose.

I think I did well in this activity because all my traversal outputs matched my handwritten results and findData worked on all three traversal methods. At first, I had a hard time with the pointer logic, especially in the constructor and in the find functions that were returning the wrong node. Fixing these helped me understand how nullptr checks and recursion work in a tree. I also want to practice tracing recursive functions by hand before coding them, so I can understand the flow more easily and catch mistakes earlier.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ClaudeAI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts, code assistance, summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used to explain concepts that helped me understand and determine which course of action to take when creating programs to satisfy requirements. It was also used in helping summarize ideas.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.